CS3281 / CS5281

Synchronization

CS3281 / CS5281

Spring 2026

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

VANDERBILT UNIVERSITY

Review

- Threads enable concurrency
  - Data sharing and non-atomic operations can lead to race conditions
- Most architectures provide atomic machine instructions that provide the building blocks for synchronization primitives
- The test-and-set instruction is an example of such an instruction; it does two things atomically
  - Obtain the previous value of a variable
  - Update the current value

Outline

- **Interrupts**
- Race Conditions
- Lock Design
- Lock Implementation
- Lock Optimization

Where else does concurrency come from?

- Processors introduce it for performance reasons by running multiple processes and threads
- Interactions with the outside world introduce it because events occur whenever they feel like it
  - Network request arriving
  - User presses a key
  - Motion sensor triggers
- Also, we need some way to deal with errors the occur when executing instructions

Interrupts

A way for the CPU to be, well, *interrupted*.

- CPU hardware switches to privileged mode
  - Now any instruction can be executed, including privileged ones.
- Execution jumps to a predefined location
  - Handler specified in the CPU's interrupt vector table
  - Lets the kernel deal with whatever the event was
- Used to support asynchronous I/O
  - Lets a hardware device tell the CPU that some data is ready
  - Remember that a disk operation is millions of times slower than an *add*.
- CPU has electrical pin(s) for hardware interrupts.
- There is also an instruction for *software* interrupts

Interrupt Vector Table

Table 6-1. Exceptions and Interrupts

Vector No.	Mnemonic	Description	Source
0	#DE	Divide Error	DIV and IDIV instructions.
1	#DB	Debug	Any code or data reference.
2		NMI Interrupt	Non-maskable external interrupt.
3	#BP	Breakpoint	INT 3 instruction.
4	#OF	Overflow	INTO instruction.
5	#BR	BOUND Range Exceeded	BOUND instruction.
6	#UD	Invalid Opcode (UnDefined Opcode)	UD2 instruction or reserved opcode.[1]
7	#NM	Device Not Available (No Math Coprocessor)	Floating-point or WAIT/FWAIT instruction.
8	#DF	Double Fault	Any instruction that can generate an exception, an NMI, or an INTR.
9	#MF	CoProcessor Segment Overrun (reserved)	Floating-point instruction.[2]
10	#TS	Invalid TSS	Task switch or TSS access.
11	#NP	Segment Not Present	Loading segment registers or accessing system segments.
12	#SS	Stack Segment Fault	Stack operations and SS register loads.
13	#GP	General Protection	Any memory reference and other protection checks.
14	#PF	Page Fault	Any memory reference.
15		Reserved	
16	#MF	Floating-Point Error (Math Fault)	Floating-point or WAIT/FWAIT instruction.
17	#AC	Alignment Check	Any data reference in memory.[3]
18	#MC	Machine Check	Error codes (if any) and source are model dependent.[4]
19	#XM	SIMD Floating-Point Exception	SIMD Floating-Point Instruction[5]
20-31		Reserved	
32-255		Maskable Interrupts	External interrupt from INTR pin or INT *n* instruction.

Table actually lives in memory somewhere, with function pointers for each vector number

Example from Tock for SAM4L chip (in Rust)

Differences

- When we performed a system call:
  - We knew it was about to happen.
  - Set up our registers in advance.
  - Performed what looked sort of like a function call.

- Interrupts might happen *whenever.*
  - This can get extremely complicated on modern systems with out-of-order execution, multiple cores and threads, and caches

Thread Scheduling in More Detail

- Time sharing is driven by the timer interrupt
  - Calls scheduler_tick(), which invokes the scheduler to see if another thread should run
- Figure on the right shows the functions of the scheduler
  - Time sharing: every runnable thread gets a chance to run; highest priority goes first
  - Preemption: the scheduler can preempt the currently running thread and run another
  - Load balancing: the scheduler can move runnable threads to other CPUS

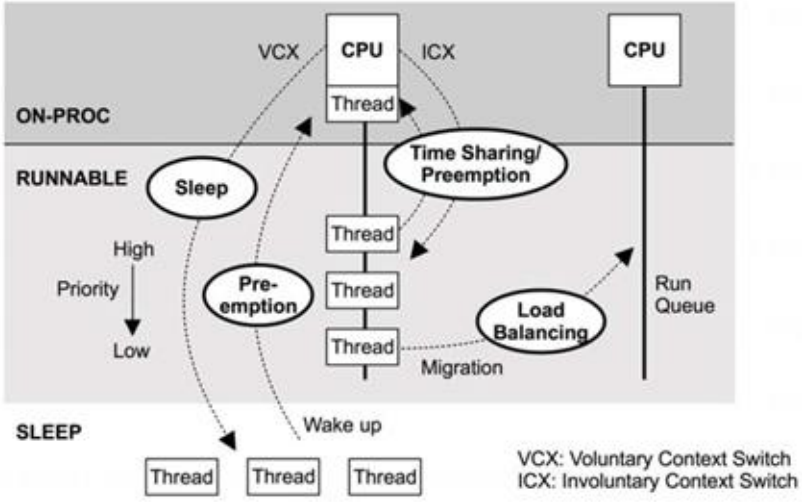

Figure 6-12 Kernel CPU scheduler functions

*Figure from *Systems Performance Enterprise and the Cloud* by Brendan Gregg

Outline

- Interrupts
- **Race Conditions**
- Lock Design
- Lock Implementation
- Lock Optimization

Concurrency can create tricky problems

```c
#include <stdio.h>
#include <pthread.h>

static volatile int counter = 0;
static const int LOOPS = 1e7;

void* mythread(void* arg) {
  printf("%s: begin\n", (char*)arg);
  for (int i=0; i<LOOPS; i++) {
    counter++;
  }
  printf("%s: done\n", (char*)arg);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  printf("main: begin (counter = %d)\n",
counter);
  pthread_create(&p1, NULL, mythread, "A");
  pthread_create(&p2, NULL, mythread, "B");

  // wait for threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: done with both (counter = %d,
goal was %d)\n", counter, 2*LOOPS);
  return 0;
}
```

- Start two threads, each of which increments a shared global **counter** variable 10^7 times.
- The **`volatile`** keyword tells the compiler that the counter variable may change unexpectedly (in this case, changed by the other thread). Don't cache it! Make sure you grab the latest value from the memory.

Concurrency can create tricky problems

```c
#include <stdio.h>
#include <pthread.h>

static volatile int counter = 0;
static const int LOOPS = 1e7;

void* mythread(void* arg) {
  printf("%s: begin\n", (char*)arg);
  for (int i=0; i<LOOPS; i++) {
    counter++;
  }
  printf("%s: done\n", (char*)arg);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  printf("main: begin (counter = %d)\n",
counter);
  pthread_create(&p1, NULL, mythread, "A");
  pthread_create(&p2, NULL, mythread, "B");

  // wait for threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: done with both (counter = %d,
goal was %d)\n", counter, 2*LOOPS);
  return 0;
}
```

- Start two threads, each of which increments a shared global **counter** variable 10^7 times.
- The **`volatile`** keyword tells the compiler that the counter variable may change unexpectedly (in this case, changed by the other thread). Don't cache it! Make sure you grab the latest value from the memory.

The increment failure in detail: 50 + 1 + 1 = 51!

Remember: each thread has its own unique registers

OS	Thread 1	Thread 2	PC	%eax	counter
			(after instruction)		
	before critical section		100	0	50
	mov 0x8049a1c, %eax		105	**50**	50
	add $0x1, %eax		108	**51**	50
interrupt					
save T1's state					
restore T2's state			100	0	50
		mov 0x8049a1c, %eax	105	**50**	50
		add $0x1, %eax	108	**51**	50
		mov %eax, 0x8049a1c	113	51	**51**
interrupt					
save T2's state					
restore T1's state			108	51	**51**
	mov %eax, 0x8049a1c		113	51	**51**

Assume the scheduler is evil

- Remember that processes have no control over the scheduler.
- So, to protect against concurrency bugs, we must assume that the scheduler can interrupt us at any time and schedule any other process.
- In other words, assume that the scheduler is ***adversarial***, and will do the worst possible scheduling.

- To prevent weird and rare concurrency bugs, your code **must** work correctly even when faced with an *evil scheduler.*

Race Condition

- Two or more things are happening at the same time
- It's not clear which will run when
- The result will be different depending on execution order
- Result becomes indeterminate (non-deterministic)

- Data race
  - Two or more threads access shared resource at the same time and at least one modifies it

Critical Section

- Code that interacts with a shared resource must not be executed concurrently
- Part of code that accesses a shared resource is a **Critical Section**
  - In other words, code that would lead to a data race
  - May be multiple, unrelated critical sections for multiple shared resources
- Critical sections need to be addressed for correctness
  - Races can be avoided by never overlapping multiple critical sections
  - We must execute critical sections “atomically” (all or none)

Critical section occurs when shared memory is accessed

```c
#include <stdio.h>
#include <pthread.h>

static volatile int counter = 0;
static const int LOOPS = 1e7;

void* mythread(void* arg) {
  printf("%s: begin\n", (char*)arg);
  for (int i=0; i<LOOPS; i++) {
    counter++;
  }
  printf("%s: done\n", (char*)arg);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  printf("main: begin (counter = %d)\n",
counter);
  pthread_create(&p1, NULL, mythread, "A");
  pthread_create(&p2, NULL, mythread, "B");

  // wait for threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: done with both (counter = %d,
goal was %d)\n", counter, 2*LOOPS);
  return 0;
}
```

When do critical sections occur?

- Critical sections often involve modification of multiple related data
  - While the modifications are happening there is some inconsistency
  - The inconsistency is eventually resolved before leaving the critical section
- For example:
  - Inserting an element in the middle of a linked list
    - Two pointers must change. List is broken if just one is changed.
  - Swapping two values.
- Don't have to worry about critical sections if:
  - Program is single-threaded, OR
  - The particular data is not shared among threads and modified, OR
  - Operation is just one assembly instruction (CPU executes these atomically)

Where is the critical section?

```c
#include <stdio.h>
#include <pthread.h>

static volatile char* person1;
static volatile char* person2;
static const int LOOPS = 1e4;

void* mythread(void* arg) {
  printf("%s: begin\n", (char*)arg);
  int i;
  for (i=0; i<LOOPS; i++) {
    // swap
    volatile char* tmp = person1;
    person1 = person2;
    person2 = tmp;
  }
  printf("%s: done\n", (char*)arg);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  person1 = "Jack";
  person2 = "Jill";
  printf("main: begin (%s, %s)\n",
         person1, person2);
  pthread_create(&p1, NULL, mythread,
"A");
  pthread_create(&p2, NULL, mythread,
"B");

  // wait for threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: end (%s, %s)\n",
         person1, person2);
}
```

Buggy concurrent swap. What can go wrong?

```c
#include <stdio.h>
#include <pthread.h>

static volatile char* person1;
static volatile char* person2;
static const int LOOPS = 1e4;

void* mythread(void* arg) {
  printf("%s: begin\n", (char*)arg);
  int i;
  for (i=0; i<LOOPS; i++) {
    // swap
    volatile char* tmp = person1;
    person1 = person2;
    person2 = tmp;
  }
  printf("%s: done\n", (char*)arg);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  person1 = "Jack";
  person2 = "Jill";
  printf("main: begin (%s, %s)\n",
         person1, person2);
  pthread_create(&p1, NULL, mythread,
"A");
  pthread_create(&p2, NULL, mythread,
"B");

  // wait for threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: end (%s, %s)\n",
         person1, person2);
}
```

Buggy concurrent swap. What can go wrong?

```c
#include <stdio.h>
#include <pthread.h>

static volatile char* person1;
static volatile char* person2;
static const int LOOPS = 1e4;

void* mythread(void* arg) {
  printf("%s: begin\n", (char*)arg);
  int i;
  for (i=0; i<LOOPS; i++) {
    // swap
    volatile char* tmp = person1;
    person1 = person2;
    person2 = tmp;
  }
  printf("%s: done\n", (char*)arg);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  person1 = "Jack";
  person2 = "Jill";
  printf("main: begin (%s, %s)\n",
         person1, person2);

  pthread_join(p2, NULL);
  printf("main: end (%s, %s)\n",
         person1, person2);
}
```

For a brief period in time:

person1: “Jill”

person2: “Jill”

Check your understanding. Is there a problem here?

```c
#include <stdio.h>
#include <pthread.h>

static volatile int sum_amount = 2;
static const int LOOPS = 1e7;

void* mythread(void* arg) {
  int counter = 0;
  printf("%s: begin\n", (char*)arg);
  for (int i=0; i<LOOPS; i++) {
    counter += sum_amount;
  }
  printf("%s: done %d\n", (char*)arg,
                                counter);
  return NULL;
}
```

```c
int main(int argc, char* argv[]) {
  pthread_t p1, p2;
  printf("main: begin\n");
  pthread_create(&p1, NULL, mythread, "A");
  pthread_create(&p2, NULL, mythread, "B");

  // wait for threads to finish
  pthread_join(p1, NULL);
  pthread_join(p2, NULL);
  printf("main: done\n");
  return 0;
}
```

Check your understanding. Is there a problem here?

```c
#include <stdio.h>
#include <pthread.h>

static volatile int sum_amount = 2;
static const int LOOPS = 1e7;

void* mythread(void* arg) {
  int counter = 0;
  printf("%s: begin\n", (char*)arg);
  for (int i=0; i<LOOPS; i++) {
    counter += sum_amount;
  }
  printf("%s: done %d\n", (char*)arg,
                                  counter);
  return NULL;
}
```

This code will work!

All threads only *read* from shared memory.

If at least one *wrote* to shared memory, it would be a problem.

Outline

- Interrupts
- Race Conditions
- **Lock Design**
- Lock Implementation
- Lock Optimization

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

Solution Requirements

We **MUST** stop data races from occurring in our programs.

1. No two processes may simultaneously be in their critical sections.

2. Processes outside of critical sections should have no impact.

3. No assumptions should be made about number of cores, speed of cores, or scheduler choices.

Locks (also known as a mutex)

- Locks are the simplest mutual exclusion primitive
  - Represent a resource that can be reserved and freed

- ***Acquire/lock***:
  - Used before a critical section to **reserve** the resource
  - If the lock is free (unlocked), then lock it and proceed.
  - If the lock is already taken (someone else called *acquire/lock*), then **wait until it's free** before proceeding.
- ***Release/unlock***:
  - Used at the end of a critical section to **free** the resource
  - Only the thread holding the lock can release it
  - Allows one waiting (or future) thread to acquire the lock

Two different metaphors & etymology

Lock

- Think about locking a bathroom door
- Our virtual lock works as follows:
  - Anyone can **lock** or **unlock** (there is no “key”).
  - Trying to enter (**lock**) if the lock is already-locked will cause you to wait until it’s unlocked.

Token

- Holding the token gives you permission to do something.
- There is only one token.
- Thus, you:
  1. Try to **acquire** the token (“lock”). You have to wait your turn if someone else is holding it.
  2. When done, **release** the token/lock.
- The token represents exclusive access to a shared resource or a critical section.

Outline

- Interrupts
- Race Conditions
- Lock Design
- **Lock Implementation**
- Lock Optimization

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

Atomic Instruction: Compare And Swap

- Example atomic_compare_and_swap (remember, this is pseudocode for hardware)

```
bool atomic_compare_and_swap (int* pointer, int expected_value, int new_value) {
    int actual_value = *pointer;
    if (actual_value == expected_value) {
        *pointer = new_value;
        return true;
     }
    return false;
}
```

- atomic_compare_and_swap(destptr, oldval, newval)
  - x86-64 instruction: `lock; cmpxchg`
  - Generalization of exchange
    - Exchange(ptr, new) -> CompareAndSwap(ptr, *ptr, new)

Mutexes

- Spin-locks are usually not the right solution for locking
  - They waste CPU cycles when the lock is contended
- Better solution is to *sleep* if the lock is not available
  - Have the OS wake you up when the lock is available
- Linux provides mutexes for just this purpose
  - Part of the POSIX library

pthread Mutexes

- Data type:
  - pthread_mutex_t
- Operations:
  - int pthread_mutex_lock(pthread_mutex_t *mutex);
  - int pthread_mutex_unlock(pthread_mutex_t *mutex);
- Typical use:

```
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

void *thread_func(void *arg)
{
  pthread_mutex_lock(&mtx); // get the lock
  // access shared data
  pthread_mutex_unlock(&mtx); // release the lock
}
```

Deadlocks with Mutexes

- Consider the following scenario

Thread A	Thread B
1. *pthread_mutex_lock(mutex1);*	1. *pthread_mutex_lock(mutex2);*
2. *pthread_mutex_lock(mutex2);* blocks	2. *pthread_mutex_lock(mutex1);* blocks

- The result is a deadlock!
- Lesson: threads should always acquire locks in the same order

*Figure from *The Linux Programming Interface* by Michael Kerrisk

Locking the Same Lock

- What if the same thread tries to obtain the same mutex multiple times?
  - The result depends on how the mutex was initialized
- Types of pthread mutexes:
  - PTHREAD_MUTEX_DEFAULT or PTHREAD_MUTEX_NORMAL
    - Results in a deadlock if the same pthread tries to lock it a second time using the pthread_mutex_lock subroutine without first unlocking it. This is the default type.
  - PTHREAD_MUTEX_ERRORCHECK
    - Avoids deadlocks by returning a non-zero value if the same thread attempts to lock the same mutex more than once without first unlocking the mutex.
  - PTHREAD_MUTEX_RECURSIVE
    - Allows the same pthread to recursively lock the mutex using the pthread_mutex_lock subroutine without resulting in a deadlock or getting a non-zero return value from pthread_mutex_lock. The same pthread has to call the pthread_mutex_unlock subroutine the same number of times as it called pthread_mutex_lock subroutine in order to unlock the mutex for other pthreads to use.

Linux Implementation of Mutexes

- Locking:
  - Bit 31: indicates if lock is taken
    - 1: taken
    - 0 free
  - Remaining bits: number of waiters
  - Line 7: don't return from here until we get the lock
  - Lines 8-10: check lock is free and decrement # of waiters if so
    - Decrement because we incremented
  - Line 15: check if lock is taken
  - Line 17: futex system call
    - Put calling process on queue

```
1  void mutex_lock (int *mutex) {
2    int v;
3    /* Bit 31 was clear, we got the mutex (the fastpath) */
4    if (atomic_bit_test_set (mutex, 31) == 0)
5      return;
6    atomic_increment (mutex);
7    while (1) {
8        if (atomic_bit_test_set (mutex, 31) == 0) {
9            atomic_decrement (mutex);
10           return;
11       }
12       /* We have to waitFirst make sure the futex value
13          we are monitoring is truly negative (locked). */
14       v = *mutex;
15       if (v >= 0)
16         continue;
17       futex_wait (mutex, v);
18   }
19 }
20
21 void mutex_unlock (int *mutex) {
22   /* Adding 0x80000000 to counter results in 0 if and
23      only if there are not other interested threads */
24   if (atomic_add_zero (mutex, 0x80000000))
25     return;
26
27   /* There are other threads waiting for this mutex,
28      wake one of them up.  */
29   futex_wake (mutex);
30 }
```

*Figure from *Operating Systems: Three Easy Pieces* by Arpaci-Dusseau and Arpaci-Dusseau

Linux Implementation of Mutexes

- Unlocking:
  - Lines 24-25: if *mutex == 0 after adding 0x80000000, then nobody was waiting
  - Line 29: invoke the futex system call
    - Argument of FUTEX_WAKE
- futex() is a multiplexed system call
  - Different arguments change the behavior
  - FUTEX_WAIT: go to sleep until mutex is available
  - FUTEX_WAKE: wake up someone waiting on this mutex
- Described in paper "Futexes are Tricky" by Ulrich Drepper

```
1  void mutex_lock (int *mutex) {
2    int v;
3    /* Bit 31 was clear, we got the mutex (the fastpath) */
4    if (atomic_bit_test_set (mutex, 31) == 0)
5      return;
6    atomic_increment (mutex);
7    while (1) {
8        if (atomic_bit_test_set (mutex, 31) == 0) {
9            atomic_decrement (mutex);
10           return;
11       }
12       /* We have to waitFirst make sure the futex value
13          we are monitoring is truly negative (locked). */
14       v = *mutex;
15       if (v >= 0)
16         continue;
17       futex_wait (mutex, v);
18   }
19 }
20
21 void mutex_unlock (int *mutex) {
22   /* Adding 0x80000000 to counter results in 0 if and
23      only if there are not other interested threads */
24   if (atomic_add_zero (mutex, 0x80000000))
25     return;
26
27   /* There are other threads waiting for this mutex,
28      wake one of them up.  */
29   futex_wake (mutex);
30 }
```

*Figure from *Operating Systems: Three Easy Pieces* by Arpaci-Dusseau and Arpaci-Dusseau

Producer-Consumer Problem

- Canonical example for condition variables: the producer-consumer problem
  - *Producer* threads produce elements
  - *Consumer* threads consume the elements produced by the producer threads
  - For example, a web server produces (provides) the web content such as HTML files and images. They are consumed (read) by the client web browser
- A lock alone isn't a good solution:
  - It only ensures mutual exclusion
  - Consider the case where a consumer wants to run but there are no elements available:
    - Obtain lock
    - Check for elements
    - Release lock
    - Sleep
- Condition variables to the rescue! They naturally express waiting for a predicate on shared state

Condition Variables

- A condition variable allows one thread to inform other threads about changes in the state of a shared variable (or other shared resource) and allows the other threads to wait (block) for such notification.
- Typical use:

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t cond_empty = PTHREAD_COND_INITIALIZER;

void *producer_func(void *arg)
{
  pthread_mutex_lock(&mtx);
  while (num_avail >= MAX_SIZE)
    pthread_cond_wait(&cond_empty, &mtx);
  num_avail++;
  pthread_cond_signal(&cond_full);
  pthread_mutex_unlock(&mtx);
}
```

Condition Variables (cont.)

- The mutex associated with a condition variable is for mutual exclusion
- The condition variable is for signaling
- Important: always check the condition in a while loop! pthread_cond_signal() means "something may have changed, go check," not "you now own a slot."

```
void *consumer_func(void *arg)
{
  pthread_mutex_lock(&mtx);
  while (num_avail <= 0)
    pthread_cond_wait(&cond_full, &mtx);
  // consume data and process
  num_avail--;
  pthread_cond_signal(&cond_empty);
  pthread_mutex_unlock(&mtx);
}
```

This atomically:
1. Unlocks the mutex
2. Waits on the condition variable

When execution reaches here, you have obtained the mutex, so you must unlock it

Condition Variable Operations

- Basic operations: signaling and waiting
  - Signaling wakes up only one waiter

- Functions return 0 on success, non-zero on error

```
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

Semaphores

- Semaphores are another synchronization primitive that can be used for both signaling and mutual exclusion
  - Originally proposed by Dijkstra
- In real life: semaphore is a system of signals for communicating visually
  - Usually with flags or lights
- Conceptually: a data type with integer value and two operations:
  - P(): decrement value of integer; block if it would go below 0
  - V(): increment the value of the integer; wake a waiting thread (if any)

P for "prolaag", a contraction of "probeer" (Dutch for "try") and "verlaag" ("decrease");
V for the Dutch word "verhoog" which means "increase"

Semaphores

- Like an integer with three differences:
  - When you create it, you can initialize its value to any integer
    - Afterwards you can only increase by one and decrease by one
  - When a thread decrements the semaphore, if the result is negative, the thread is blocked and cannot continue until another thread increments the semaphore
  - When a thread increments the semaphore, if other threads are waiting, one waiting thread gets unblocked
- This implies:
  - There's no way to know if a thread will block before it decrements a semaphore
  - After a thread increments a semaphore, you don't know whether it or waiting thread that woke up will continue running
  - When you increment a semaphore, you don't know whether there is any waiting thread

Semaphores

- Values:
  - Positive: number of threads that can decrement without blocking
  - Negative (conceptually): number of threads that are blocked and waiting
  - Zero: No resources available, trying to decrement will block
- A mutex can be implemented as a binary semaphore
  - Initialize the semaphore to 1
- “Counting” problems can initialize the semaphore to an arbitrary value
- Linux implementation: can be used across processes
  - Mutexes and condition variables only usable between threads of same process
- Data type: sem_t and associated functions
  - Similar to POSIX mutexes and condition variables

Rendezvous

- Common synchronization pattern
- Two threads (A and B)
  - A has to wait for B
  - B has to wait for A
- In other words:
  - a1 happens before b2
  - b1 happens before a2

Thread A

```
statement a1
statement a2
```

Thread B

```
statement b1
statement b2
```

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

Rendezvous Solution

- Common synchronization pattern
- Two threads (A and B)
  - A has to wait for B
  - B has to wait for A
- In other words:
  - a1 happens before b2
  - b1 happens before a2
- Solution on the right
  - Tell the other thread you've arrived
  - Then wait on the other thread

Thread A

```
statement a1
aArrived.signal()
bArrived.wait()
statement a2
```

Thread B

```
statement b1
bArrived.signal()
aArrived.wait()
statement b2
```

While working on the previous problem, you might have tried something like this:

Barriers

- Another common synchronization pattern
  - "Generalizes" the Rendezvous to arbitrary number of threads
- In other words, no thread reaches the critical point until everyone has executed the rendezvous

Barrier code

```
rendezvous
critical point
```

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

Barriers Solution

```
pthread_mutex_lock(&mtx);

arrived++;
if (arrived == N) {
    // last one: release everybody
    pthread_cond_broadcast(&cv);
} else {
    while (arrived < N) {   // MUST be while
        pthread_cond_wait(&cv, &mtx);  // atomically unlock+sleep, then relock on wake
    }
}

pthread_mutex_unlock(&mtx);
```

Summary: Synchronization Implementation

- How are all of these synchronization primitives implemented?
- If no sleeping involved: with atomic machine instructions
  - Example: spin-locks use the atomic compare and exchange instruction
- If sleeping involved: with the *futex* system call
- With pthreads, the mutex, condition variable, and semaphore operations are all defined in the C library
  - "Fast" path (i.e., the lock is free): no need to transition to kernel-space (i.e., no system call)
  - "Slow" path (i.e., lock is contended): the C library invokes the futex system call

Synchronization Implementation (cont.)

- How is the futex system call implemented?
  - Quickly check again if the resource is free
  - If not, the kernel puts the calling process on a queue associated with the lock/semaphore/condition variable
  - The kernel then switches to another process
    - Putting the process on the queue and switching to another needs to happen atomically so a wake-up isn't missed;
      - Corner cases often complicate the implementation
- When another thread/process releases the lock:
  - Check if there's anyone on the queue associated with that lock
  - If so, wake them up

Outline

- Interrupts
- Race Conditions
- Lock Design
- Lock Implementation
- **Lock Optimization**

Evaluating a lock

Requirements for correctness

- Mutual Exclusion:
  - Only one thread in critical section at a time
- Progress (deadlock-free):
  - If several simultaneous requests, must allow one to proceed
- Bounded Wait (starvation-free):
  - Must eventually allow every waiting thread to proceed

Additional goals

- Fairness – each thread waits for the same amount of time
- Performance – do the above in minimal execution time